DBMS MINI PROJECT

SplitEase

Personal Expense Splitting App

The complete guide: every concept explained from zero, the database design, every query, the website, and a ready-made presentation script with viva questions and answers.

Item	Details
Database	MongoDB 8 (NoSQL document database)
Database language	JavaScript in mongosh (MongoDB Shell): schema, data and queries
Website (frontend)	Next.js 16 + React 19, connected with the official MongoDB Node.js driver
Tools	Homebrew, Node.js and npm, mongosh, MongoDB Compass, Terminal (macOS)
Project files	expense_splitter.js (database script) · expense-app/ (website) · ER diagrams · report

How to use this guide

If you have a day: read Parts 1 to 5 in order. They build up from zero, with no prior knowledge assumed.

If you have an hour: read the Quick Summary (next page), then Part 7 (presentation script) and Part 8 (viva Q&A).

Green boxes explain things **in simple words**. Amber boxes are **viva tips**: things examiners often ask.

Contents

Contents 2

Quick summary (read this first) 4

Part 1 — The basics, from zero 5

1.1 What is a database? . . . 5

1.2 Two families of databases: SQL and NoSQL . . . 5

1.3 MongoDB words you must know . . . 5

1.4 The same expense in SQL and in MongoDB . . . 6

1.5 Frontend, backend and database . . . 6

Part 2 — The problem and the requirements 7

2.1 The problem . . . 7

2.2 What the database must store (the 5 entities) . . . 7

2.3 What the database must do (functional requirements) . . . 7

2.4 The sample data used everywhere in this guide . . . 7

Part 3 — Database design 8

3.1 The ER diagram (conceptual design) . . . 8

3.2 From ER diagram to MongoDB (logical design) . . . 10

Part 4 — Building the database 12

4.1 The four collections and their fields . . . 12

4.2 Schema validation: making MongoDB strict . . . 12

4.3 Indexes: making searches fast . . . 13

4.4 Business rules (the logic) . . . 13

4.5 The balance formula (the heart of the project) . . . 14

Part 5 — The four queries, explained step by step 15

5.1 What is an aggregation pipeline? . . . 15

5.2 Query 1 — All expenses and total spending of a group . . . 15

5.3 Query 2 — How much each member owes or is owed . . . 16

5.4 Query 3 — Expense history and contributions of one user . . . 17

5.5 Query 4 — Unsettled balances and recording payments . . . 17

Part 6 — The website (frontend) 19

6.1 How the pieces fit together . . . 19

6.2 The pages . . . 19

6.3 What happens when you click “Add expense” . . . 21

6.4 The website's files . . . 22

Part 7 — Software used, how to run it, and testing 23

7.1 Software and why each one was used . . . 23

7.2 How to run everything (Mac) . . . 23

7.3 Testing that was done 24
Part 8 — Presentation script (slide by slide) 25
Live demo script (about 2 minutes) 27
Part 9 — Viva questions and answers 28
Basics 28
Design 28
Queries and indexes 29
Logic 30
Website and tools 30
Part 10 — Glossary 32

Quick summary (read this first)

What is it? SplitEase is a database project for people who share costs (flatmates, friends on a trip, families). It records who paid for what, works out who owes whom, and records the payments that settle those debts.

What did we build?

- A **MongoDB database** with 4 collections: **users, groups, expenses, settlements**, with rules (validation) and indexes.
- The **4 required queries**, written as MongoDB aggregation pipelines, plus sample data to show them working.
- A **website** (Next.js) where you can see everything and add people, groups, expenses and payments.
- **ER diagrams**, a report, and this guide.

The 4 questions the database answers:

#	Question	Example answer from sample data
1	All expenses and total spending of a group	Flat 302 spent ₹47,998.00 across 6 expenses
2	How much each member owes or is owed	Aarav gets back ₹23,946.41; Rohan owes ₹10,404.58
3	Expense history and contributions of one user	Aarav: in 9 expenses, paid ₹47,200.00, own share ₹19,568.59
4	Unsettled balances, and recording payments	Goa trip settled with 3 payments, all to Karthik

The 3 most important design ideas (memorise these):

1. **Embedding:** the list of people sharing an expense is stored *inside* the expense document, and a group's member list is stored *inside* the group document. One read gets everything.
2. **Balances are calculated, never stored.** They are worked out from expenses and payments every time, so they can never be wrong or out of date. In every group they add up to exactly ₹0.00.
3. **Rules are enforced twice:** by MongoDB itself ($jsonSchema validation and a unique index) and by the application (for example, custom shares must add up to the total).

> **One-sentence pitch for the start of your presentation**
>
> “SplitEase is a MongoDB-based expense splitting system that stores shared expenses as documents, calculates each person’s balance with aggregation pipelines, and suggests the fewest payments needed to settle up, all shown through a Next.js website.”

Part 1 — The basics, from zero

1.1 What is a database?

A **database** is an organised place to store information so it can be saved, searched and updated reliably. A **DBMS** (Database Management System) is the software that manages it. Examples: MySQL, PostgreSQL, Oracle, MongoDB.

Without a database, an expense app would keep data in files, and it would be easy to lose data, save half an update, or have two copies that disagree. A DBMS prevents this with rules, indexes and safe writes.

> **In simple words**
>
> A database is like a very strict, very fast notebook. It only lets you write things in the right format, finds anything instantly, and never loses a page.

1.2 Two families of databases: SQL and NoSQL

	SQL (relational)	NoSQL (document) — our choice
Examples	MySQL, PostgreSQL, Oracle	MongoDB, CouchDB, Firestore
Data stored as	Tables with rows and columns	Documents (like JSON) inside collections
Structure	Fixed columns, defined first	Flexible; can still be enforced with validation
Related data	Split into tables, joined with JOIN	Often kept together in one document (embedding)
Language	SQL	MongoDB Query Language and aggregation pipelines (JavaScript-like)
Scaling	Usually a bigger server	Designed to spread across many servers (sharding)

1.3 MongoDB words you must know

MongoDB word	SQL equivalent	Meaning
Database	Database	A container for collections. Ours is called **expense_splitter**.
Collection	Table	A group of similar documents, e.g. **users**.
Document	Row	One record, written like JSON: `{ name: "Aarav", email: "…" }`
Field	Column	One piece of data inside a document, e.g. **email**.
_id	Primary key	Every document's unique ID, created automatically.
ObjectId	Auto ID	The 12-byte type of _id, e.g. ObjectId('6ae032b4…'). It also contains the creation time.
Embedded document / array	Child table	A document or list stored inside another document.
Reference	Foreign key	Storing another document's _id to link to it, e.g. **paidBy**.
$jsonSchema validator	CHECK / NOT NULL	Rules a document must follow before it is saved.

MongoDB word	SQL equivalent	Meaning
Index	Index	A sorted lookup structure that makes searches fast.
Aggregation pipeline	SELECT … JOIN … GROUP BY	A series of stages that filter, join, group and calculate.
mongosh	mysql client	The command-line program that talks to MongoDB.

1.4 The same expense in SQL and in MongoDB

This is the single most useful picture for understanding the project. On the left, a relational database needs three tables for one expense. On the right, MongoDB keeps the whole expense in one document.

```
users            expenses                       expense_participants
-------          --------------------------     --------------------
id | name        id | desc     | amount | ...   expense_id | user_id | share
1  | Aarav       7  | Groceries| 1849          7           | 1       | 616.34
2  | Priya                                     7           | 2       | 616.33
3  | Rohan                                     7           | 3       | 616.33

→ 3 tables, and a JOIN is needed to read one expense
```

```
{
  _id: ObjectId("6ae0…1d"),
  description: "Groceries",
  amount: 1849,
  paidBy: ObjectId("…0c"),        // Rohan
  splitType: "EQUAL",
  participants: [
    { userId: ObjectId("…0a"), share: 616.34 },
    { userId: ObjectId("…0b"), share: 616.33 },
    { userId: ObjectId("…0c"), share: 616.33 }
  ]
}
→ 1 document holds the whole expense
```

> **Viva tip**
>
> If asked “why MongoDB?”, point at this example: an expense and its participants are always read and written together, so storing them as one document is simpler and faster, and saving one document is atomic (it either fully succeeds or fully fails).

1.5 Frontend, backend and database

- **Frontend:** what the user sees and clicks (web pages in the browser).
- **Backend:** code on the server that receives requests, applies rules, and talks to the database.
- **Database:** where the data is permanently stored (MongoDB).

In our website, **Next.js** does both frontend and backend in one project: it builds the pages and also runs the server code that queries MongoDB.

Part 2 — The problem and the requirements

2.1 The problem

When people share costs, three things go wrong: they forget who paid, splitting is not always equal (someone skips a meal), and it is hard to know the smallest set of payments that settles everything. A shared notebook or a WhatsApp chat does not solve this.

2.2 What the database must store (the 5 entities)

Entity	Meaning	Example
User	A person using the app	Aarav Sharma, aarav@example.com
Group	A set of people who share costs	Flat 302 - Koramangala; Goa Trip 2026
Expense	Something one person paid for the group	October rent ₹40,000 paid by Aarav
Expense Participant	Who shares an expense, and their share	Rent: Aarav ₹10,000, Priya ₹10,000, …
Settlement / Payment	Money paid back to clear a debt	Sneha paid Aarav ₹5,000 by UPI

2.3 What the database must do (functional requirements)

1. A user can create or join **multiple groups**.
2. Add shared expenses and choose **who is involved**.
3. Split an expense **equally** or by **custom amounts**.
4. Maintain each user's **balance** (owes / is owed).
5. Record **payments** that settle debts.
6. **Query 1:** show all expenses and total spending for a selected group.
7. **Query 2:** calculate how much each member owes or is owed.
8. **Query 3:** show the expense history and contributions of a particular user.
9. **Query 4:** identify unsettled balances and record settlement payments.

2.4 The sample data used everywhere in this guide

The database script loads realistic data so every query has something to show:

Group	Members	Expenses
Flat 302 - Koramangala monthly flat expenses	Aarav (admin), Priya, Rohan, Sneha	Rent ₹40,000 (Aarav) · Electricity ₹2,350 (Priya) · Groceries ₹1,849, Sneha away (Rohan) · Wi-Fi ₹999 (Sneha) · Pizza ₹1,600, custom split (Priya) · Cleaning ₹1,200 (Aarav)
Goa Trip 2026 4-day trip	Karthik (admin), Aarav, Meera, Rohan (joined later)	Resort ₹18,000 (Karthik) · Scooter ₹2,400, custom (Meera) · Dinner ₹3,260 (Rohan) · Water sports ₹6,000, custom (Aarav)

Two earlier payments are also loaded: Sneha → Aarav ₹5,000 (UPI) and Rohan → Karthik ₹2,000 (Cash). Aarav and Rohan are in **both** groups, which shows that a user can be in many groups.

Part 3 — Database design

3.1 The ER diagram (conceptual design)

An **ER (Entity–Relationship) diagram** is a picture of the data before we decide how to store it. It shows the things (entities), their properties (attributes), and how they are connected (relationships).

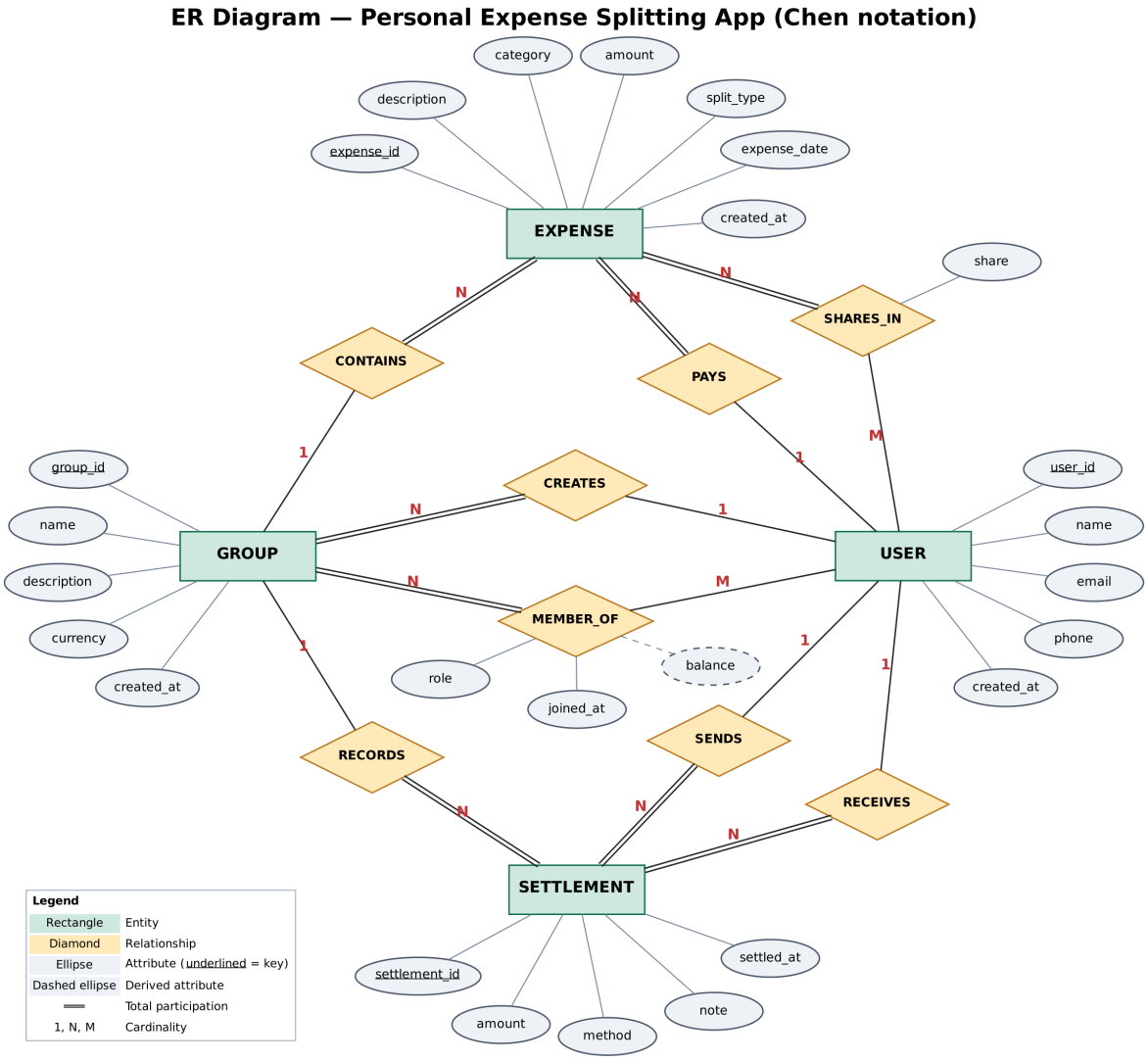


Figure 1 — ER diagram in Chen notation

How to read it

Symbol	Means	In our diagram
Rectangle	Entity (a thing we store)	USER, GROUP, EXPENSE, SETTLEMENT
Diamond	Relationship (how entities connect)	CREATES, MEMBER_OF, PAYS, SHARES_IN, …
Ellipse	Attribute (a property)	name, email, amount, …
Underlined attribute	Key: uniquely identifies the entity	user_id, group_id, expense_id, settlement_id
Dashed ellipse	Derived attribute: calculated, not stored	balance (on MEMBER_OF)

Symbol	Means	In our diagram
Double line	Total participation: every one must take part	Every expense must belong to a group
1, N, M	Cardinality: how many on each side	1 group CONTAINS N expenses

The 8 relationships explained

Relationship	Cardinality	Meaning in plain words
USER — CREATES — GROUP	1 : N	One user can create many groups; each group has exactly one creator (its admin).
USER — MEMBER_OF — GROUP	M : N	A user can be in many groups; a group has many users. Has its own attributes: role, joined_at. Every group has at least one member.
GROUP — CONTAINS — EXPENSE	1 : N	A group has many expenses; each expense belongs to one group.
USER — PAYS — EXPENSE	1 : N	One person pays for each expense; a person can pay for many.
USER — SHARES_IN — EXPENSE	M : N	An expense is shared by many people; a person shares many expenses. Has its own attribute: share (their part of the bill).
GROUP — RECORDS — SETTLEMENT	1 : N	A group has many payments; each payment belongs to one group.
USER — SENDS — SETTLEMENT	1 : N	The person who pays back money (the debtor).
USER — RECEIVES — SETTLEMENT	1 : N	The person who receives the money (the creditor).

Viva tip

Q: Which relationships are many-to-many? MEMBER_OF and SHARES_IN. In SQL each would need a separate junction table. In MongoDB we stored them as **embedded arrays**: groups.members[] and expenses.participants[].

Q: Why is balance a derived attribute? Because it can always be calculated from expenses and payments, so storing it would risk it becoming wrong.

3.2 From ER diagram to MongoDB (logical design)

Next we decide **how** to store each part. MongoDB gives two options for related data:

Option	How it works	Use it when…	We used it for
Embedding	Put the related data inside the parent document	The data is small, always read with the parent, and not needed on its own	Group members (groups.members[]) Expense participants (expenses.participants[])
Referencing	Store only the other document's _id (like a foreign key)	The data grows without limit or is queried on its own	groupId, paidBy, paidTo, createdBy, userId

So the **5 entities become 4 collections**: Group Member is embedded in **groups**, and Expense Participant is embedded in **expenses**.

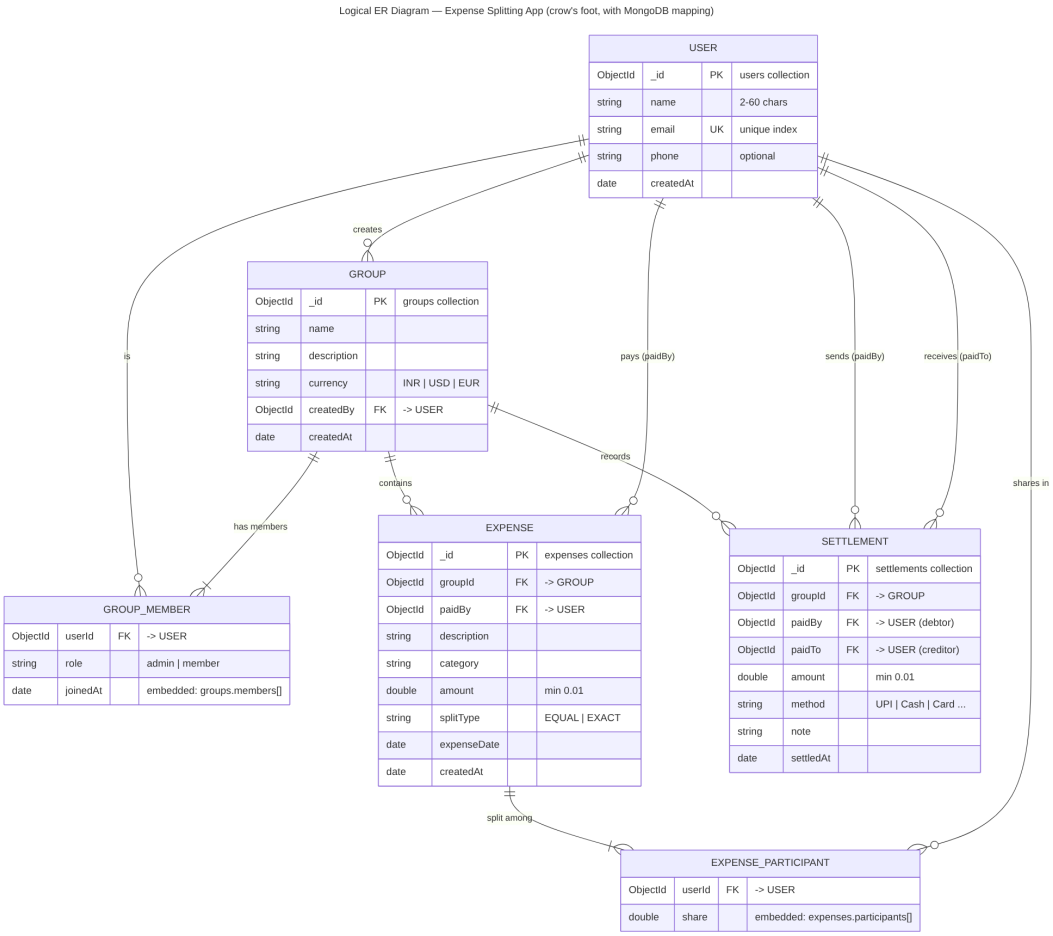


Figure 2 — Logical diagram (crow's foot) with data types, keys and MongoDB mapping

> **In simple words**
>
> Embedding is like writing the guest list on the invitation itself. Referencing is like writing “see guest list #7”. We embed small lists that always travel with their parent, and reference big, independent things.

Why not embed expenses inside groups too?

A group can have thousands of expenses over years, MongoDB documents have a 16 MB size limit, and we often query expenses on their own (for example, a user's history across all groups). So expenses get their own collection and point to their group with **groupId**.

Part 4 — Building the database

Everything in this part is in one file, **expense_splitter.js**, which you run with `mongosh expense_splitter.js`. It has 8 sections:

Section	What it does
1. Collections	Deletes the old database and creates the 4 collections with validation rules
2. Indexes	Creates 7 indexes for fast searching
3. Helper functions	addUser, createGroup, joinGroup, addExpense, equalShares, recordSettlement, suggestSettlements
4. Sample data	Inserts 6 users, 2 groups, 10 expenses, 2 payments; proves bad data is rejected
5–8. Queries	Query 1, 2, 3 and 4 (4a to 4d), printed as tables

4.1 The four collections and their fields

Collection	Fields (★ = required)	Rules enforced by MongoDB
users	_id, name★, email★, phone, createdAt★	name 2–60 characters; email must look like an email; phone 7–15 digits; email unique
groups	_id, name★, description, currency, createdBy★, members[]★ {userId, role, joinedAt}, createdAt★	at least 1 member; role is admin or member; currency INR / USD / EUR
expenses	_id, groupId★, description★, category, amount★, paidBy★, splitType★, participants[]★ {userId, share}, expenseDate★, createdAt★	amount ≥ 0.01; splitType EQUAL or EXACT; at least 1 participant; share ≥ 0; category from a fixed list
settlements	_id, groupId★, paidBy★, paidTo★, amount★, method, note, settledAt★	amount ≥ 0.01; method UPI / Cash / Bank Transfer / Card / Other

4.2 Schema validation: making MongoDB strict

MongoDB is flexible by default, so it would accept any shape of document. We switched on a **$jsonSchema validator** for each collection, so wrong data is refused by the database itself. Here is part of the expenses validator:

```
db.createCollection("expenses", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["groupId", "description", "amount", "paidBy",
               "splitType", "participants", "expenseDate", "createdAt"],
    properties: {
      amount:       { bsonType: ["double", "int", "decimal"], minimum: 0.01 },
      splitType:    { enum: ["EQUAL", "EXACT"] },
      participants: { bsonType: "array", minItems: 1, items: {
                        bsonType: "object", required: ["userId", "share"],
                        properties: { share: { minimum: 0 } } } }
    } } }
});
```

Proof it works: the script tries to insert an expense with amount −50, split type “HALF” and no participants. MongoDB refuses it with *“Document failed validation”*. It also tries to add a second user with aarav@example.com and gets error *E11000 duplicate key* from the unique

index.

4.3 Indexes: making searches fast

An **index** is like the index at the back of a textbook. Without it, MongoDB reads every document to find matches (a “collection scan”). With it, MongoDB jumps straight to the right documents.

Collection	Index	Type	Why
users	{ email: 1 }	Unique	Fast login look-up; stops two accounts with one email
groups	{ "members.userId": 1 }	Multikey	Find all groups a user belongs to
expenses	{ groupId: 1, expenseDate: -1 }	Compound	Queries 1 and 2: one group's expenses, newest first
expenses	{ paidBy: 1 }	Single field	Query 3: expenses a user paid for
expenses	{ "participants.userId": 1 }	Multikey	Query 3: expenses a user shares in
settlements	{ groupId: 1, settledAt: -1 }	Compound	Queries 2 and 4: a group's payments
settlements	{ paidBy: 1, paidTo: 1 }	Compound	Query 3: a user's payments

Viva tip

Multikey index = an index on a field inside an array (like participants.userId). MongoDB makes one index entry per array element, so “find every expense Priya shares in” is fast. **Compound index** = an index on two fields together; the order matters (groupId first, because we always filter by group).

4.4 Business rules (the logic)

Some rules are too complex for a schema, so the helper functions (and the website) check them before saving:

Rule	Why
Payer and every participant must be members of the group	You can't split a flat bill with someone outside the flat
Equal split is calculated in paise (whole numbers)	So shares always add up exactly, with no rounding error
Custom (EXACT) shares must add up to the expense amount	Otherwise money would appear or disappear
A payment must go from someone who owes to someone who is owed	Prevents nonsense payments
A payment cannot be more than the amount outstanding	Prevents over-paying and flipping balances
Nobody can pay themselves	Meaningless transaction

Worked example: equal split with rounding

Groceries ₹1,849 split equally among 3 people. 1849 ÷ 3 = 616.333..., which can't be paid in paise. The function converts to paise: 184,900 ÷ 3 = 61,633 each, with 1 paisa left over. The first person gets the extra paisa:

Person	Share
Aarav	₹616.34
Priya	₹616.33
Rohan	₹616.33
Total	**₹1,849.00** ✓

```
function equalShares(amount, userIds) {
  const paise = Math.round(amount * 100);              // 184900
  const base  = Math.floor(paise / userIds.length);    // 61633
  let extra   = paise - base * userIds.length;         // 1 paisa left over
  return userIds.map(id => ({ userId: id, share: (base + (extra-- > 0 ? 1 : 0)) / 100 }));
}
```

4.5 The balance formula (the heart of the project)

For every member of a group:

net balance = (amount paid for expenses + payments sent) − (own share of expenses + payments received)

If net is **positive**, the member **gets money back**. If **negative**, the member **owes**. If zero, they are **settled up**. Every rupee one person is owed is owed by someone else, so **all nets in a group always add up to 0**; the script checks this every time.

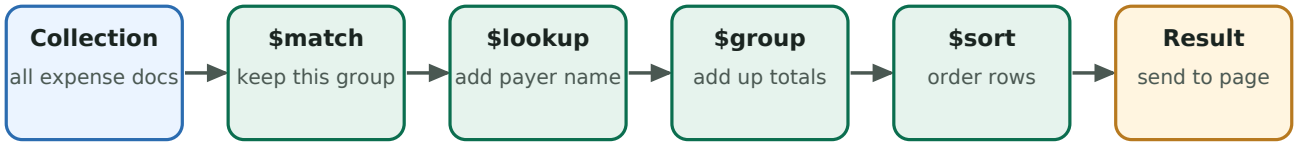
Worked example: Aarav in Flat 302

Item	Amount
Paid for expenses: rent 40,000 + cleaning 1,200	+ ₹41,200.00
Payments sent	+ ₹0.00
Own share: rent 10,000 + electricity 587.50 + groceries 616.34 + Wi-Fi 249.75 + pizza 500 + cleaning 300	− ₹12,253.59
Payments received (Sneha paid him)	− ₹5,000.00
Net balance	**+ ₹23,946.41 → gets back**

Part 5 — The four queries, explained step by step

5.1 What is an aggregation pipeline?

All four queries use MongoDB's **aggregation pipeline**. Think of it as an **assembly line**: documents go in one end, pass through stages, and each stage changes them. The result comes out the other end.



Documents flow left to right; each stage changes them and hands them on.

Stage	What it does	SQL equivalent
$match	Keep only documents that match a condition	WHERE
$lookup	Bring in matching documents from another collection	LEFT OUTER JOIN
$unwind	Turn one document with an array into one document per array item	(no direct equivalent)
$group	Combine documents and calculate totals ($sum, $avg)	GROUP BY with SUM(), AVG()
$project / $set	Choose, rename or calculate fields	SELECT column list / expressions
$sort	Order the results	ORDER BY
$facet	Run several pipelines on the same input at once	Several queries in one
$unionWith	Add documents from another collection to the stream	UNION ALL
$filter / $map	Pick or transform items inside an array	(subquery on child table)

5.2 Query 1 — All expenses and total spending of a group

Goal: list a group's expenses with who paid, plus the total, the average, and a breakdown by category.

```
db.expenses.aggregate([
  { $match:  { groupId: flat } },                                  // 1. only this group
  { $lookup: { from: "users", localField: "paidBy",                // 2. fetch payer's name
               foreignField: "_id", as: "payer" } },
  { $set:    { paidByName: { $arrayElemAt: ["$payer.name", 0] },
               splitAmong: { $size: "$participants" } } },         // 3. how many share it
  { $sort:   { expenseDate: 1 } },                                 // 4. oldest first
  { $facet: {                                                      // 5. three answers at once
      expenses:   [ { $project: { description: 1, amount: 1, paidByName: 1, … } } ],
      total:      [ { $group: { _id: null, totalSpending: { $sum: "$amount" },
                                expenseCount: { $sum: 1 }, avgExpense: { $avg: "$amount" } } } ],
      byCategory: [ { $group: { _id: "$category", spent: { $sum: "$amount" } } },
                    { $sort: { spent: -1 } } ] } }
])
```

Date	Description	Category	Amount	Paid by	Split
01 Oct	October rent	Rent	₹40,000.00	Aarav	Equal, 4
05 Oct	Electricity bill	Utilities	₹2,350.00	Priya	Equal, 4
08 Oct	Groceries (Sneha away)	Groceries	₹1,849.00	Rohan	Equal, 3
10 Oct	Wi-Fi recharge	Utilities	₹999.00	Sneha	Equal, 4
12 Oct	Pizza night	Food	₹1,600.00	Priya	Custom, 4
15 Oct	Deep cleaning service	Services	₹1,200.00	Aarav	Equal, 4

Result: total spending **₹47,998.00** across 6 expenses, average ₹7,999.67. By category: Rent 40,000 · Utilities 3,349 · Groceries 1,849 · Food 1,600 · Services 1,200.

5.3 Query 2 — How much each member owes or is owed

Goal: apply the balance formula from 4.5 to every member, inside the database. The trick is to turn every expense and payment into small **ledger entries** (like rows in an account book), then add them up per person.

1. **$match** the group's expenses.
2. **$project + $unwind:** each expense becomes one entry for the payer (*paid = amount*) and one entry per participant (*share = their share*).
3. **$unionWith settlements:** each payment adds an entry for the sender (*sent*) and one for the receiver (*received*).
4. **$unionWith groups:** adds a zero entry for every member, so someone with no activity still appears.
5. **$group by userId:** adds up paid, share, sent and received for each person.
6. **$set net** = (paid + sent) − (share + received), rounded to 2 decimals.
7. **$lookup** the user's name, then **$sort** by net (biggest creditor first).

```
{ $project: { entries: { $concatArrays: [
    [ { userId: "$paidBy", paid: "$amount", share: 0, sent: 0, received: 0 } ],
    { $map: { input: "$participants", as: "p",
              in: { userId: "$$p.userId", paid: 0, share: "$$p.share", sent: 0, received: 0 } } }
] } } },
{ $unwind: "$entries" }, { $replaceWith: "$entries" },
{ $unionWith: { coll: "settlements",
                pipeline: [ /* +sent for payer, +received for receiver */ ] } },
{ $group: { _id: "$userId", totalPaid: { $sum: "$paid" }, totalShare: { $sum: "$share" },
            settledOut: { $sum: "$sent" }, settledIn: { $sum: "$received" } } },
{ $set: { net: { $round: [ { $subtract: [ { $add: ["$totalPaid", "$settledOut"] },
                                          { $add: ["$totalShare", "$settledIn"] } ] }, 2 ] } } }
```

Member (Flat 302)	Paid	Own share	Sent	Received	Net	Status
Aarav Sharma	41,200.00	12,253.59	0.00	5,000.00	+23,946.41	gets back
Sneha Reddy	999.00	11,437.25	5,000.00	0.00	−5,438.25	owes
Priya Nair	3,950.00	12,053.58	0.00	0.00	−8,103.58	owes
Rohan Mehta	1,849.00	12,253.58	0.00	0.00	−10,404.58	owes

Check: 23,946.41 − 5,438.25 − 8,103.58 − 10,404.58 = **0.00** ✓

5.4 Query 3 — Expense history and contributions of one user

Goal: for one person, list every expense they paid for or share in, show what they paid and what their share was, and total it per group.

```
db.expenses.aggregate([
  { $match: { $or: [ { paidBy: u }, { "participants.userId": u } ] } },   // uses 2 indexes
  { $set: {
      myShare: { $sum: { $map: {
        input: { $filter: { input: "$participants", as: "p",
                            cond: { $eq: ["$$p.userId", u] } } },        // pick my entry
        as: "p", in: "$$p.share" } } },
      iPaid: { $cond: [ { $eq: ["$paidBy", u] }, "$amount", 0 ] } } },
  { $set: { netEffect: { $subtract: ["$iPaid", "$myShare"] } } },
  { $lookup: { from: "groups", localField: "groupId", foreignField: "_id", as: "g" } },
  { $facet: { history: [ … ], perGroup: [ { $group: … } ], overall: [ { $group: … } ] } }
])
```

Result for Aarav: involved in **9 expenses** across 2 groups, paid for 3 of them. He paid **₹47,200.00** against an own share of **₹19,568.59**. Per group: Flat 302 (paid 41,200.00, share 12,253.59) and Goa Trip (paid 6,000.00, share 7,315.00). He also received ₹5,000 from Sneha.

5.5 Query 4 — Unsettled balances and recording payments

4a. Unsettled balances: run Query 2 for every group and keep anyone whose net is not zero. Before settling, 8 balances are open (4 in each group).

4b. Suggest the fewest payments. A simple **greedy algorithm**: repeatedly match the person who owes the most with the person who is owed the most, and pay the smaller of the two amounts. This needs at most (members − 1) payments.

Worked example — Goa Trip 2026

Step	Biggest debtor	Biggest creditor	Payment	Left over
Start	Meera −5,515 · Aarav −1,315 · Rohan −1,255	Karthik +8,085		
1	Meera owes 5,515	Karthik is owed 8,085	Meera → Karthik ₹5,515	Karthik still owed 2,570
2	Aarav owes 1,315	Karthik is owed 2,570	Aarav → Karthik ₹1,315	Karthik still owed 1,255
3	Rohan owes 1,255	Karthik is owed 1,255	Rohan → Karthik ₹1,255	Everyone at 0 ✓

4c. Recording a payment inserts a document into **settlements** after checking the rules (payer owes, receiver is owed, amount not more than outstanding, not paying yourself):

```
recordSettlement(goa, meera, karthik, 5515, "UPI", "Trip settle-up");
// inserts: { groupId, paidBy: meera, paidTo: karthik, amount: 5515,
//            method: "UPI", note: "Trip settle-up", settledAt: new Date() }
```

After the 3 payments every Goa member shows ₹0.00. A further ₹100 payment is correctly **rejected**. **4d** then lists the group's payment history.

Viva tip

Because balances are derived, recording a payment needs only **one insert**. No balance field anywhere has to be updated, so there is nothing to forget or get out of sync.

Part 6 — The website (frontend)

The database answers the queries; the website lets a normal person use them. It is called **SplitEase** and is built with **Next.js**. It reads and writes the same **expense_splitter** database.

6.1 How the pieces fit together

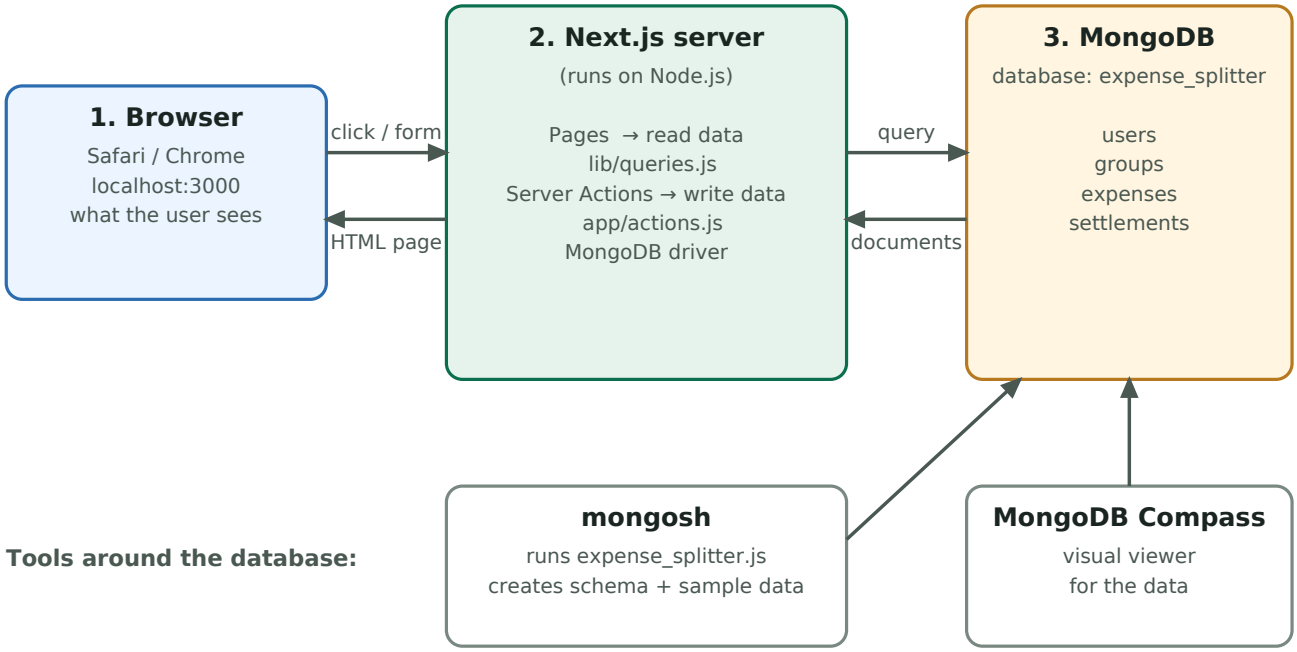


Figure 3 — System architecture

1. You open **localhost:3000** in the browser.
2. The Next.js server runs a **page** (for example, the group page), which calls the queries in **lib/queries.js**.
3. Those queries are the same aggregation pipelines as in Part 5, sent to MongoDB by the **MongoDB Node.js driver**.
4. MongoDB returns documents; Next.js turns them into an HTML page and sends it to the browser.
5. When you submit a form, a **Server Action** in **app/actions.js** checks the rules and inserts the document. The page then refreshes with the new numbers.

6.2 The pages

Page	Address	What you can do	Query
Dashboard	/	See totals, all groups and all unsettled balances; create a group	1, 4a
Group	/groups/[id]	See expenses and total, spending by category, balances, settle-up suggestions, payment history; add an expense, record a payment, add a member, delete an expense	1, 2, 4
People	/users	See everyone; add a person	—
Person	/users/[id]	See one person's expense history, contribution per group and payments	3

Every section on the site has a small green tag such as **Query 2**, so during the demo you can point to exactly which requirement it answers.

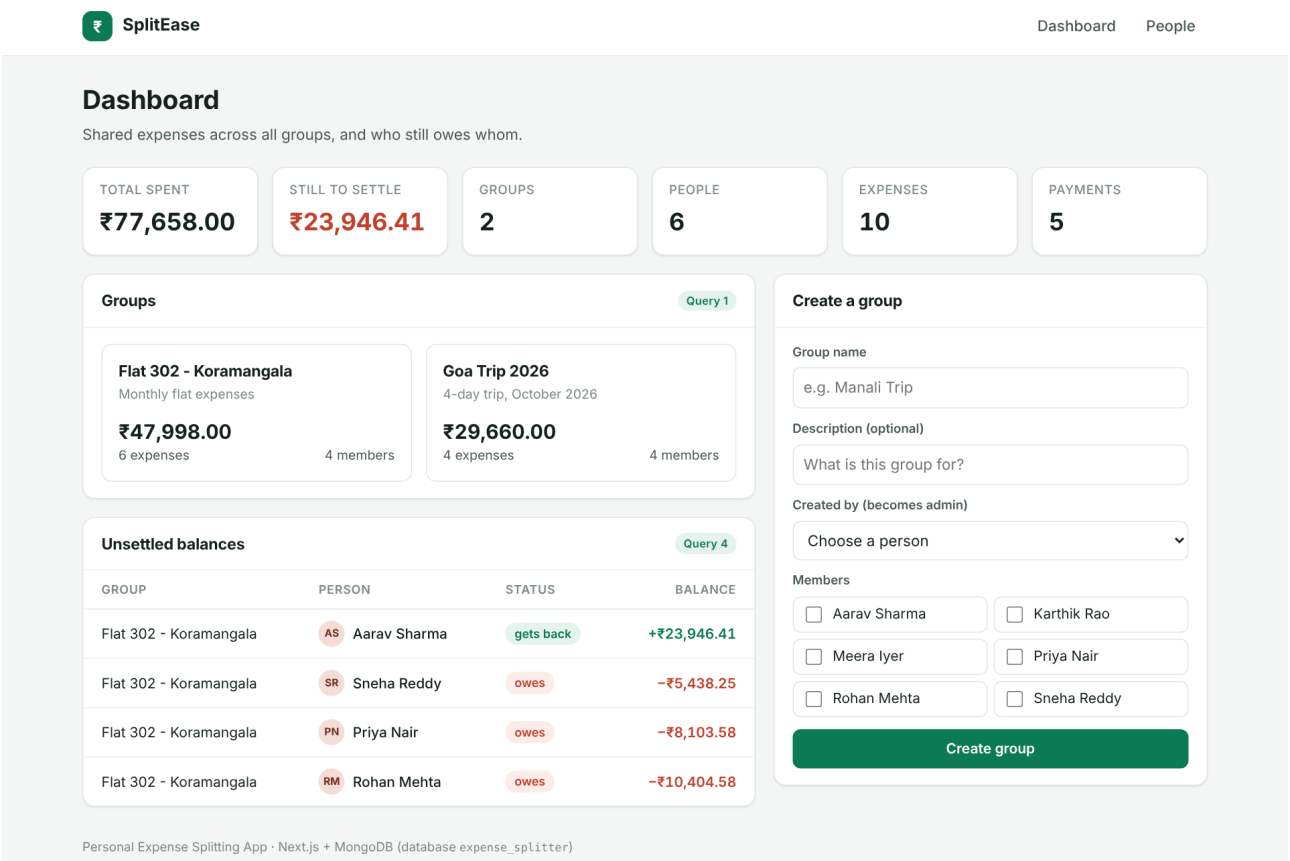


Figure 4 — Dashboard: overall totals, groups (Query 1) and unsettled balances (Query 4a)

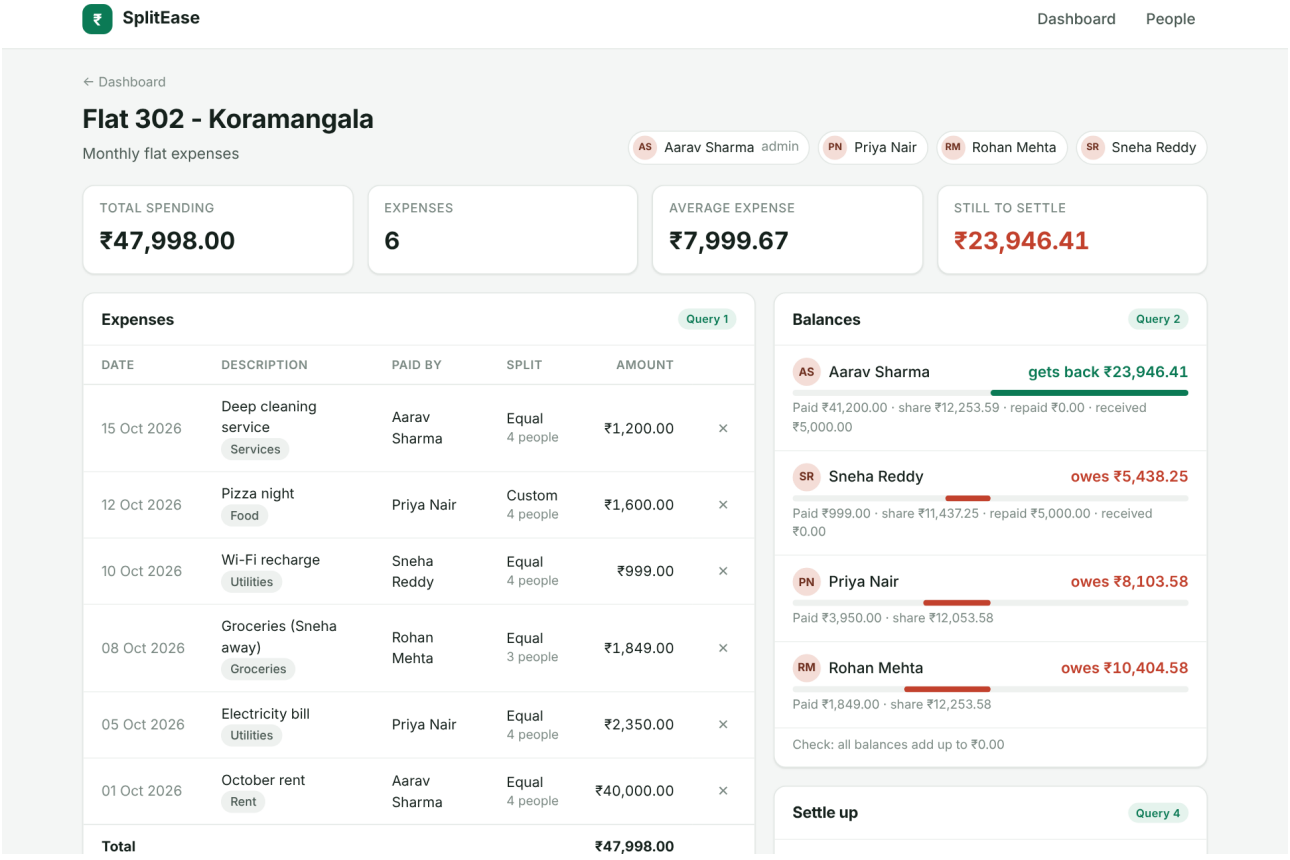


Figure 5 — Group page: expenses and total (Query 1) next to member balances (Query 2)

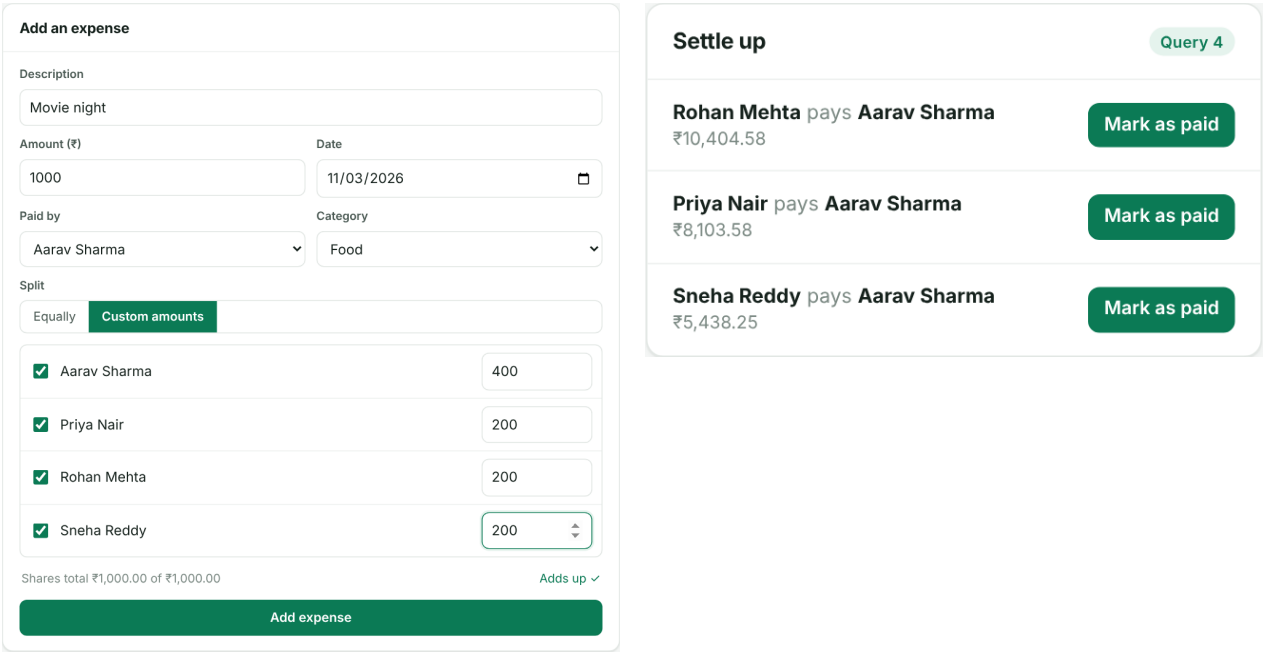


Figure 6 — Left: adding an expense with a custom split; the form checks that shares add up. Right: settle-up suggestions with a one-click “Mark as paid” (Query 4)

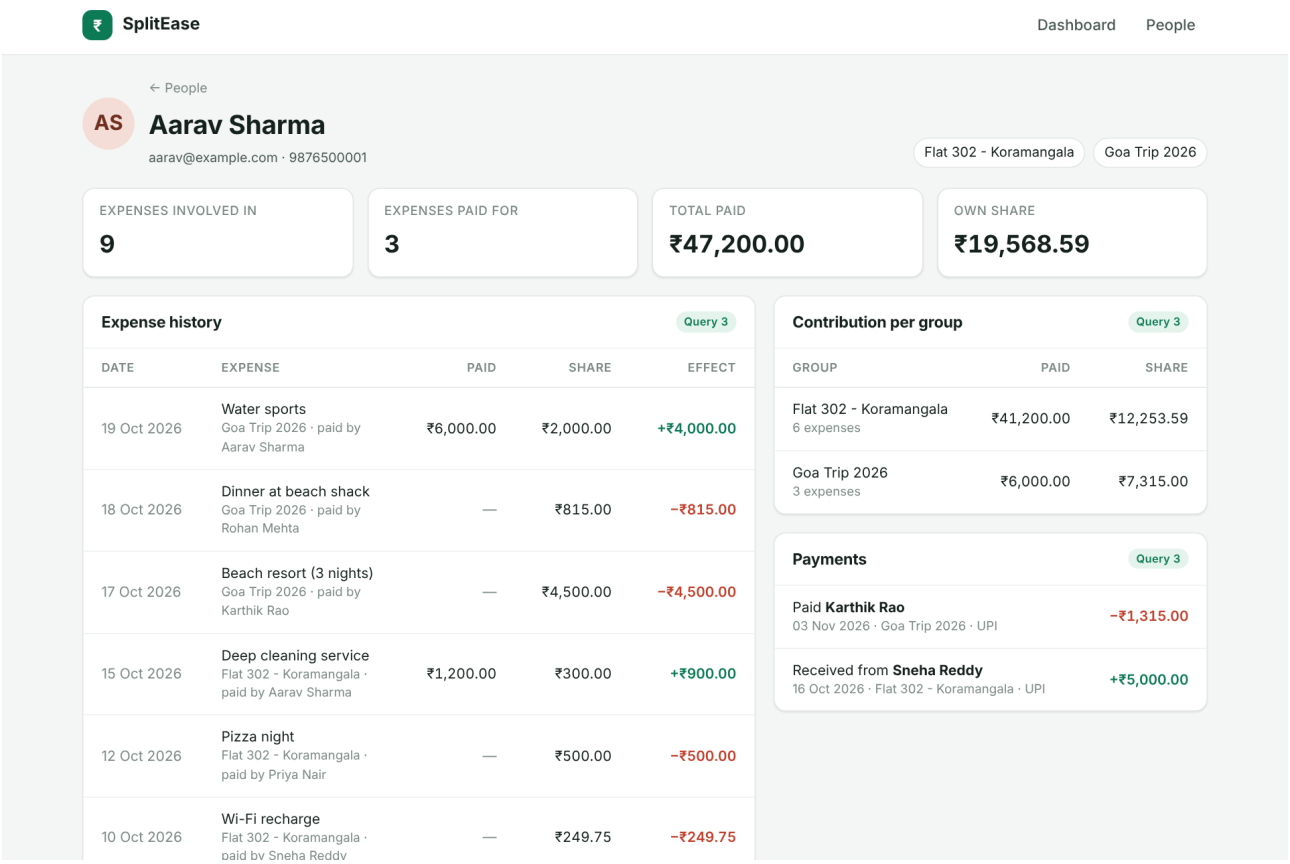


Figure 7 — Person page: expense history and contributions (Query 3)

6.3 What happens when you click “Add expense”

Step	Where	What happens
1	Browser (Forms.jsx)	The form collects description, amount, payer, date, split type and participants. For a custom split it shows a live “Adds up ✓” check.

Step	Where	What happens
2	Server (actions.js → addExpense)	Checks: description present, amount ≥ ₹0.01, at least one participant, custom shares add up, payer and participants are group members. Equal splits are calculated in paise.
3	MongoDB	insertOne into expenses. The $jsonSchema validator checks the document again.
4	Server	revalidatePath tells Next.js the data changed.
5	Browser	The page re-renders: new expense in the list, new total, new balances, new settle-up suggestions. On an error, a red message shows and your typing is kept.

6.4 The website's files

File	Purpose
lib/mongodb.js	Opens one shared connection to mongodb://127.0.0.1:27017 and re-uses it
lib/queries.js	All read queries: the Part 5 pipelines (group report, balances, user history, settlements)
app/actions.js	All writes (Server Actions): createUser, createGroup, addMember, addExpense, deleteExpense, recordSettlement
components/Forms.jsx	The interactive forms (runs in the browser)
components/Ui.jsx	Small display pieces: avatars, cards, the Query tags
app/page.js, app/groups/[id]/page.js, app/users/…	The pages
app/globals.css	The design: colours, layout, light and dark mode
package.json	Lists the packages (next, react, mongodb) and the commands (seed, dev)
expense_splitter.js	A copy of the database script, run by npm run seed

Part 7 — Software used, how to run it, and testing

7.1 Software and why each one was used

Software	What it is	Why we used it
MongoDB Community Server 8	The database server	Required by the project; documents fit expenses with participant lists
mongosh	MongoDB's command-line shell (runs JavaScript)	Runs the whole database script; lets you type queries by hand in a viva
MongoDB Compass	MongoDB's free visual app	Shows collections and documents visually, including the embedded arrays
Node.js and npm	JavaScript runtime and its package manager	Next.js runs on Node.js; npm downloads Next.js, React and the MongoDB driver
Next.js 16 + React 19	A web framework for building websites	Builds pages and server code in one project; Server Actions make database writes simple and safe
MongoDB Node.js driver	The official library that connects code to MongoDB	Lets the website run the same queries as mongosh
Homebrew	The macOS package manager	Installs MongoDB, mongosh and Node.js with single commands
Terminal	The Mac's command-line app	Where all the commands are typed

7.2 How to run everything (Mac)

Every time (Terminal):

```
brew services start mongodb-community          # 1. start the database
cd ~/Downloads/expense-app                      # 2. go to the project folder
npm run seed                                    # 3. (only to reset) load sample data
npm run dev                                     # 4. start the website
# then open http://localhost:3000 in Safari or Chrome
```

First time only, before the above: `brew install mongosh node` and then `npm install` inside the folder. To view data visually, open MongoDB Compass and connect to `mongodb://127.0.0.1:27017`.

Problem	Fix
Website says “Can't connect to MongoDB”	brew services start mongodb-community
mongosh: command not found	brew install mongosh
Refusing to load ... untrusted tap mongodb/brew	brew trust mongodb/brew
A group link shows “Not found”	You re-seeded, so IDs changed; go back to the dashboard
Port 3000 already in use	npm run dev -- -p 3001, then open localhost:3001

7.3 Testing that was done

Test	Result
Ran the full database script on a real MongoDB 8.3 server	All 4 queries correct; every group's balances sum to 0.00
Inserted an invalid expense (negative amount, bad split type, no participants)	Rejected: Document failed validation
Inserted a duplicate email	Rejected: E11000 duplicate key
Recorded a payment larger than what was owed	Rejected with a clear message
Website: added an expense whose custom shares didn't add up	Rejected; fixed it and it saved; total and balances updated
Website: equal split of ₹100 among 3	₹33.34 + 33.33 + 33.33; balances still sum to 0.00
Website: clicked every “Mark as paid” suggestion	Group shows everyone settled up
Website: created a group, added a person and a member, deleted an expense	All worked; delete asks for confirmation first
Website on a phone-sized screen and in dark mode	No layout overflow; readable in both themes

Part 8 — Presentation script (slide by slide)

A 10–12 minute talk in 12 slides. For each slide: what to **put on it** and what to **say**. Speak slowly; the numbers in brackets are approximate times.

1. Title (0:30)

On the slide	SplitEase — Personal Expense Splitting App · DBMS mini project · team names · MongoDB + Next.js
Say	“Good morning. Our project is SplitEase, a database system that helps friends, flatmates and families split shared expenses and settle up. We built the database in MongoDB and a website on top of it in Next.js.”

2. The problem (1:00)

On the slide	Who paid? Who owes? Unequal splits. Too many small payments. (A picture of a messy group chat works well.)
Say	“When people share costs, it's hard to remember who paid, splits are often unequal, and nobody knows the simplest way to settle. We wanted a database that answers these questions accurately.”

3. Objectives and requirements (1:00)

On the slide	5 entities: User, Group, Expense, Expense Participant, Settlement. 4 queries: group spending, balances, user history, unsettled balances and payments.
Say	“The database stores five entities and must answer four key queries. A user can be in many groups, and an expense can be split equally or with custom amounts.”

4. Technology stack (0:45)

On the slide	MongoDB · mongosh · MongoDB Compass · Next.js + React · Node.js · Homebrew
Say	“MongoDB is the database. We used mongosh to create the schema and run the queries, Compass to view the data, and Next.js for the website.”

5. ER diagram (1:30)

On the slide	Figure 1 (Chen notation).
Say	“Here are our four main entities and eight relationships. MEMBER_OF and SHARES_IN are many-to-many. Balance is a dashed, derived attribute, because it is calculated, not stored. Double lines show total participation: every expense must belong to a group.”

6. Mapping to MongoDB (1:15)

On the slide	Figure 2, or the SQL-vs-document example from Part 1.4. Embedding vs referencing.
Say	“The two many-to-many relationships became embedded arrays: members inside groups, and participants inside expenses. So five entities became four collections. We embed small lists that are always read with their parent, and reference large or independent data.”

7. Validation and indexes (1:00)

On the slide	$jsonSchema rules · unique email · 7 indexes (compound and multikey).
Say	“MongoDB is flexible, so we added schema validation: a negative amount or a wrong split type is refused. A unique index stops duplicate emails. Seven indexes make every query fast.”

8. Queries 1 and 2 (1:30)

On the slide	The pipeline picture · Flat 302 total ₹47,998 · balances table · “sum = 0”.
Say	“Our queries use aggregation pipelines: an assembly line of stages like match, lookup and group. Query 1 gives a group's expenses and total. Query 2 turns every expense and payment into ledger entries and adds them up per person. The balances always add up to zero.”

9. Queries 3 and 4 (1:15)

On the slide	Aarav's history · the Goa settle-up table (3 payments).
Say	“Query 3 shows one user's history across groups. Query 4 lists unsettled balances and suggests the fewest payments using a greedy algorithm. For the Goa trip, three payments settle everyone. Recording a payment is just one insert.”

10. Live demo (2:00)

On the slide	Switch to the browser (see the demo script on the next page).
Say	“Let me show it working.” Then follow the demo script.

11. Testing and results (0:45)

On the slide	The testing table from Part 7.3 (pick 4–5 rows).
Say	“We tested on a real MongoDB server. Invalid data and duplicates are rejected, over-payments are refused, and balances always net to zero.”

12. Conclusion and future scope (0:45)

On the slide	Achieved all 4 queries · future: transactions, percentage splits, multi-currency, recurring expenses, receipts, mobile app. “Thank you, questions?”
Say	“To conclude, we built a working MongoDB database and website that meet all the requirements. In future we could add multi-document transactions, more split types and a mobile app. Thank you. We're happy to take questions.”

Live demo script (about 2 minutes)

Before the presentation: start MongoDB, run `npm run seed`, start `npm run dev`, open localhost:3000 and keep MongoDB Compass open in another window.

1. **Dashboard:** “Two groups, total spent, and the unsettled balances table: that's Query 4a.”
2. Click **Flat 302**: “Here is Query 1: every expense and the total of ₹47,998. Next to it, Query 2: Aarav gets back ₹23,946.41, and the check at the bottom shows the balances add up to zero.”
3. **Add an expense:** Movie night, ₹1,000, paid by Aarav, Custom amounts 400/200/200/200. “The form checks the shares add up.” Click Add: “The total and every balance updated instantly.”
4. Try a **wrong split** (e.g. change one share to 100): “The database rule rejects it with a clear message.”
5. **Settle up:** click one “Mark as paid”: “That inserted one settlement document; the balances recalculated.”
6. Click a person, e.g. **Aarav**: “Query 3: his history across both groups and what he contributed to each.”
7. Switch to **Compass**, open expenses, and expand participants: “This is the embedded array we designed: the expense and its split live in one document.”

> **If something goes wrong in the demo**
>
> Stay calm and switch to your screenshots (Figures 4–7) or the printed output of `mongosh expense_splitter.js`. Most demo failures are just MongoDB not running: `brew services start mongodb-community`.

Part 9 — Viva questions and answers

Basics

Q. What is your project in one line?

A MongoDB database (with a Next.js website) that records shared expenses, calculates who owes whom, and records payments to settle debts.

Q. What is a DBMS?

Software that stores, organises and manages data reliably, with rules, indexes and safe concurrent access. MongoDB is our DBMS.

Q. What is the difference between SQL and NoSQL?

SQL databases store data in fixed tables and use JOINs; NoSQL document databases like MongoDB store flexible JSON-like documents and often keep related data together.

Q. What is a document and a collection?

A document is one record in JSON-like form (BSON). A collection is a group of documents, like a table. We have 4 collections.

Q. What is BSON?

Binary JSON, the format MongoDB stores documents in. It adds types JSON lacks, such as ObjectId, Date and Decimal.

Q. What is ObjectId?

MongoDB's default 12-byte unique ID for _id. It includes a timestamp, so IDs roughly sort by creation time.

Design

Q. Why did you choose MongoDB?

The project required it, and it fits: an expense and its participants are always used together, so one document holds the whole expense. Writes to one document are atomic, aggregation handles the calculations, and it can scale out by sharding.

Q. What are your entities and collections?

5 entities: User, Group, Expense, Expense Participant, Settlement. 4 collections: users, groups, expenses, settlements. Participants and members are embedded.

Q. What is embedding vs referencing?

Embedding stores related data inside the parent document; referencing stores another document's _id. We embed small, always-together lists and reference large or independent data.

Q. Why are participants embedded?

The list is small (a few people), always read with the expense, never queried alone, and must be saved together with the expense.

Q. Why are expenses not embedded inside groups?

A group's expenses grow without limit (documents max out at 16 MB), and we query expenses on their own, e.g. a user's history across groups.

Q. How do you represent many-to-many relationships?

As embedded arrays of references: groups.members[].userId (users↔groups) and expenses.participants[].userId (users↔expenses). In SQL these would be junction tables.

Q. Is MongoDB schema-less? How did you enforce structure?

It is flexible by default, but we added $jsonSchema validators: required fields, data types, allowed values (enum) and minimum amounts. Invalid documents are rejected.

Q. What are your primary and foreign keys?

Each collection's _id is the primary key. References like groupId, paidBy, paidTo and createdBy act as foreign keys. MongoDB doesn't enforce foreign keys, so our code checks membership before saving.

Q. Why don't you store the balance?

It is a derived attribute. Calculating it from expenses and payments every time means it can never become inconsistent, and a payment is just one insert.

Q. What normal form is your design in?

The equivalent relational design (users, groups, group_members, expenses, expense_participants, settlements) is in 3NF. In MongoDB we deliberately embedded the two junction tables for faster reads. There is no duplicated data, because names are looked up, not copied.

Queries and indexes

Q. What is an aggregation pipeline?

A sequence of stages ($match, $lookup, $group, $sort…) that process documents one stage after another, like an assembly line, to produce a result.

Q. What does $lookup do?

It joins documents from another collection, like a LEFT OUTER JOIN. We use it to get user names and group names.

Q. What do $unionWith and $facet do?

$unionWith appends documents from another collection (like UNION ALL); we use it to add settlements into the balance ledger. $facet runs several sub-pipelines on the same data at once; we use it to get the expense list, totals and categories in one query.

Q. Explain how Query 2 calculates balances.

Each expense becomes ledger entries: +amount for the payer and +share for each participant. Settlements add sent and received entries. $group sums them per user, and net = (paid + sent) − (share + received).

Q. How do you know your balances are correct?

In every group the nets must sum to zero, because every rupee owed is owed to someone. The script and the website both check and show this.

Q. What indexes did you create and why?

Unique on users.email; multikey on groups.members.userId; compound on expenses (groupId, expenseDate); expenses.paidBy; multikey on expenses.participants.userId; compound on settlements (groupId, settledAt) and (paidBy, paidTo). Each one supports a specific query filter.

Q. What is a multikey index?

An index on a field inside an array; MongoDB creates one entry per array element. It makes “all expenses Priya shares in” fast.

Q. What is a compound index and why does order matter?

An index on several fields. MongoDB can use its leading fields alone, so groupId comes first because every group query filters on it.

Logic

Q. How do you handle rounding in an equal split?

We split in paise (whole numbers) and give leftover paise to the first participants, e.g. ₹1,849 / 3 = 616.34 + 616.33 + 616.33.

Q. How is a custom split validated?

The shares must add up exactly to the expense amount; otherwise the expense is rejected with a message showing the difference.

Q. How does the settle-up suggestion work?

A greedy algorithm: repeatedly match the largest debtor with the largest creditor and pay the smaller amount. It needs at most (members − 1) payments.

Q. What if someone tries to pay more than they owe?

recordSettlement rejects it: the amount can't exceed the smaller of what the payer owes and what the receiver is owed.

Q. What happens to balances if an expense is deleted?

Nothing needs updating: balances are recalculated from the remaining data the next time they're shown.

Q. Does MongoDB support ACID transactions?

Yes. Single-document writes are always atomic, which is why embedding participants matters. Multi-document transactions are supported on replica sets; we list them as future scope.

Website and tools

Q. What is Next.js and why use it?

A React-based web framework. It renders pages on the server and supports Server Actions, so the website and its database code live in one project.

Q. How does the website connect to MongoDB?

Through the official MongoDB Node.js driver using the connection string mongodb://127.0.0.1:27017 and the database expense_splitter. One connection is shared across requests.

Q. What is a Server Action?

A function that runs on the server when a form is submitted. Ours validate the input, insert into MongoDB, and refresh the page data.

Q. What happens if MongoDB is not running?

The website shows a friendly error page explaining how to start it (brew services start mongodb-community).

Q. What are mongosh and Compass?

mongosh is MongoDB's command-line shell, used to run our script and queries. Compass is MongoDB's graphical tool for browsing data.

Q. What are the limitations and future scope?

Single currency per group, no login or authentication, and no multi-document transactions. Future: transactions, percentage splits, multi-currency, recurring expenses, receipt images (GridFS), notifications, and a mobile app.

Part 10 — Glossary

Term	Meaning
Aggregation pipeline	A series of stages that process documents to compute results.
Atomic	All-or-nothing: an operation either fully happens or not at all.
BSON	Binary JSON: how MongoDB stores documents.
Cardinality	How many of one entity relate to another (1:1, 1:N, M:N).
Collection	A group of documents (like a table).
Compound index	An index on more than one field.
Creditor / Debtor	The person who is owed money / the person who owes money.
Derived attribute	A value calculated from other data rather than stored (our balance).
Document	One record in MongoDB, written like JSON.
Embedding	Storing related data inside a parent document.
Entity	A thing we store data about (User, Group, …).
ER diagram	A drawing of entities, attributes and relationships.
Greedy algorithm	Solves a problem by making the best-looking choice at each step (biggest debtor pays biggest creditor).
Homebrew	A tool that installs software on a Mac from the Terminal.
Index	A data structure that makes searches fast.
$jsonSchema	MongoDB's way of defining validation rules for documents.
Ledger	A list of money movements, added up to find balances.
mongosh	MongoDB's command-line shell.
Multikey index	An index on a field inside an array.
Next.js	A React framework for building websites with server-side code.
Node.js / npm	Runs JavaScript outside the browser / installs JavaScript packages.
ObjectId	MongoDB's default unique ID type.
Referencing	Linking documents by storing another document's _id.
Server Action	A Next.js function that runs on the server when a form is submitted.
Sharding	Splitting a database across many servers to scale.
Total participation	Every instance of an entity must take part in a relationship (double line in ER).
Validation	Rules that data must pass before it is saved.

Last advice

Know three things by heart: **(1)** the 4 collections and why two lists are embedded, **(2)** the balance formula and that balances sum to zero, **(3)** what $match, $lookup and $group do. Almost every viva question leads back to one of these.

Good luck!